Merge and Code Review

Paulo Borba
Informatics Center

pauloborba.cin.ufpe.br

Interference and semantic conflicts

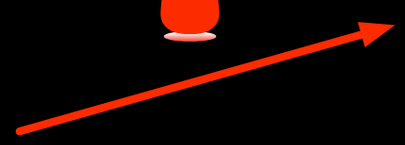
There are code integration problems beyond unnecessary merge conflicts…

Build conflicts

(static semantics/syntactic conflicts)



```
class Text {
  public String text;
  …
  void cleanText() {
    removeComments();
  }
}
```



```
class Text {
  public String text;
  …
  void normalizeWhitespace() {…}

  void cleanText() {
    removeComments();
  }
}
```



```
class Text {
  public String text;
  …
  void normalizeWhitespace() {…}

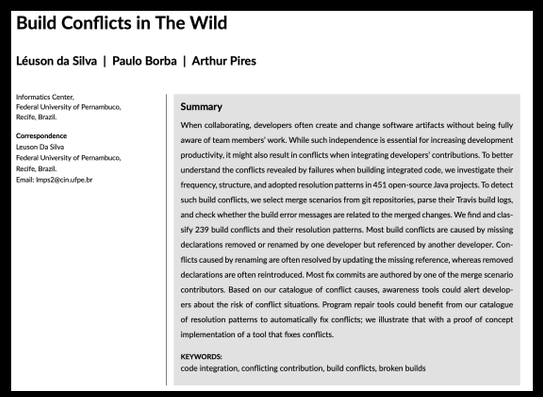
  void cleanText() {
    removeComments();
    normalizeWhitespace();
  }
}
```



merge

```
class Text {
  public String text;
  …
  void cleanText() {
    removeComments();
    normalizeWhitespace();
  }
}
```

https://pauloborba.cin.ufpe.br/publication/2022build_conflicts_in_the_wild/2022-Silva-Build conflicts in the wild.pdf



Build Conflicts in The Wild

Léuson da Silva | Paulo Borba | Arthur Pires

Informatics Center,
Federal University of Pernambuco,
Recife, Brazil.

Correspondence
Leuson Da Silva
Federal University of Pernambuco,
Recife, Brazil.
Email: lmps2@cin.ufpe.br

Summary

When collaborating, developers often create and change software artifacts without being fully aware of team members' work. While such independence is essential for increasing development productivity, it might also result in conflicts when integrating developers' contributions. To better understand the conflicts revealed by failures when building integrated code, we investigate their frequency, structure, and adopted resolution patterns in 451 open-source Java projects. To detect such build conflicts, we select merge scenarios from git repositories, parse their Travis build logs, and check whether the build error messages are related to the merged changes. We find and classify 239 build conflicts and their resolution patterns. Most build conflicts are caused by missing declarations removed or renamed by one developer but referenced by another developer. Conflicts caused by renaming are often resolved by updating the missing reference, whereas removed declarations are often reintroduced. Most fix commits are authored by one of the merge scenario contributors. Based on our catalogue of conflict causes, awareness tools could alert developers about the risk of conflict situations. Program repair tools could benefit from our catalogue of resolution patterns to automatically fix conflicts; we illustrate that with a proof of concept implementation of a tool that fixes conflicts.

KEYWORDS:
code integration, conflicting contribution, build conflicts, broken builds

Understanding and automatically resolving build conflicts

(57,065 merges from 451 Java projects with Travis CI)

6 conflict patterns

unavailable symbol is the most common (65%)

17 resolutions patterns

build conflict repair tool

covering 3 patterns

https://is.gd/TJnNcc

L. Da Silva. Detecting, Understanding and Resolving Build and Test Conflicts. ICSE Poster 2019.

L. Da Silva, P. Borba , A. Pires. Build Conflicts in The Wild. Submitted for publication, 2020.

These are caused by
syntactic or static
semantics incompatibilities

But not as bad as dynamic
semantic incompatibilities
(test or production conflicts)

Test or production conflicts

(dynamic semantics conflicts)



Detecting Semantic Conflicts via Automated Behavior Change Detection

Leuson Da Silva*, Paulo Borba*, Wardah Mahmood+, Thorsten Berger+, and João Moisakis*

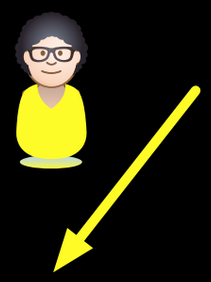
*Federal University of Pernambuco, Recife, Brazil
+Chalmers | University of Gothenburg, Gothenburg, Sweden

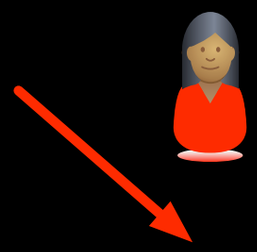
***Abstract*—Branching and merging are common practices in collaborative software development. They increase developer productivity by fostering teamwork, allowing developers to independently contribute to a software project. Despite such benefits, branching and merging comes at a cost—the need to merge software and to resolve merge conflicts, which often occur in practice. While modern merge techniques, such as 3-way or structured merge, can resolve many such conflicts automatically,**

changes to be merged. This can negatively affect development productivity, and even compromise software quality in case developers incorrectly fix conflicts [17], [6], [18]. To avoid dealing with merge conflicts, developers sometimes even adopt risky practices, such as rushing to finish changes first [19], [17] and partial check-ins [20]. Similarly, partially motivated by the need to reduce merge conflicts, development teams have been

https://pauloborba.cin.ufpe.br/publication/2020detecting_semantic_conflicts_via_automated_behavior_change_detection/

```
class Text {
  public String text;
  …
  void cleanText() {
    removeComments();
  }
}
```





```
class Text {
  public String text;
  …
  void cleanText() {
    normalizeWhitespace();
    removeComments();
  }
}
```

```
class Text {
  public String text;
  …
  void cleanText() {
    removeComments();
    removeDuplicateWords();
  }
}
```

merge

```
class Text {
  public String text;
  …
  void cleanText() {
    normalizeWhitespace();
    removeComments();
    removeDuplicateWords();
  }
}
```

```
class Text {
   public String text;
   …
   void cleanText() {
      normalizeWhitespace();
      removeComments();
      removeDuplicateWords();
   }
}
```

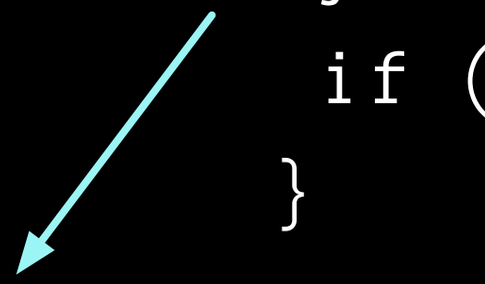
~~resulting text has no duplicate whitespace~~

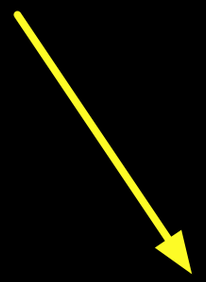
resulting text has no duplicate words

```
Text t = new Text();
t.text = "the␣the␣␣dog";
t.cleanText();
assertTrue(t.noDuplicateWhiteSpace()); FAILS!
```

```
m(x){
 if (x==null) return;
 y = x;
 if (x!=null) y = x.n();
}
```



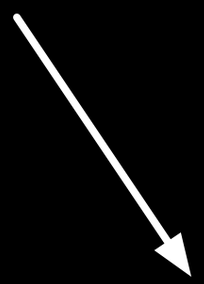


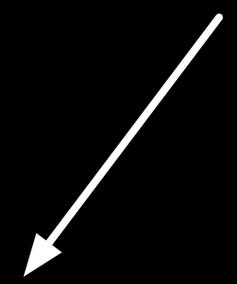




```
m(x){
 if (x==null) return;
 y = x;
 y = x.n();
}
```

```
m(x){
  y = x;
  if (x!=null) y = x.n();
}
```





```
git merge
 m(x){
  y = x;
  y = x.n();
 }
```



Semantic code integration conflict

contribution of developer F

contribution of developer G

merged code

```
x++;
y=f(x);
x--;
print(x+y);
```

previously existing code

testing or execution issues!

Conflict terminology: process x language aspects

- merge conflict — textual conflict
- build conflict — static semantics/syntactic conflict
- test conflict — dynamic/behavioral semantics conflict
- production conflict — dynamic/behavioral semantics conflict

Conflict terminology

- merge conflict—not clear how to put together textual changes—textual conflict (but not quite that for semistructured merge tools, and eventually for semantic merge tools; so classification valid only for line based merge tools) (for s3m: not clear how to put together textual changes, or clear but they break some rule that is checked—duplicated declarations—and that would lead to a build issue)

- build conflict—clear how to put together textual changes, but they lead to a build error—often due to a static semantics incompatibility, but could also be a syntactic incompatibility (one guy removed a try-catch, the other added a finally after an existing comment)

- test conflict—clear how to put together textual changes, they don’t lead to a build error, but lead to a project test error— due to a dynamic/behavioral semantics incompatibility

  production conflict (conflict revealed as a bug, escaped defect)—clear how to put together textual changes, they don’t lead to a build error, they don't lead to a project test error, but lead to an untested execution error— due to a dynamic/behavioural semantics incompatibility

dependences
but no
coordination

knowledge
breakdown

integration
conflicts

tedious and
risky resolution

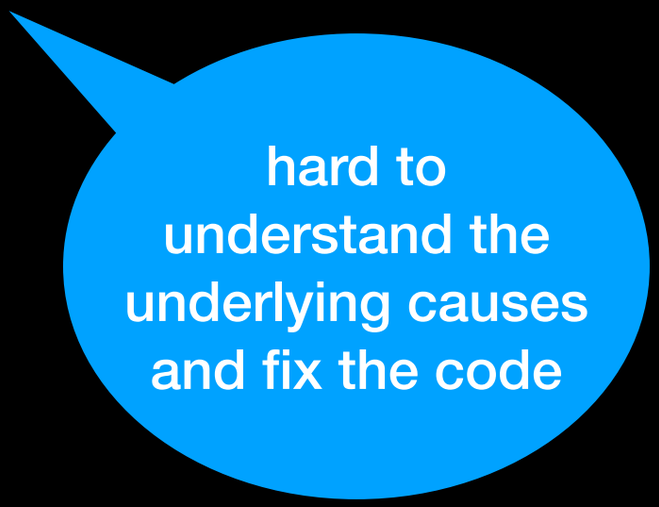
syntactic,
static semantics

hard to find
and fix bugs

dynamic semantics

Current merge tools are oblivious to the semantics of the code changes that they integrate

Missed conflicts are hardly detected by project tests or code reviews, and end up escaping to system users



We need semantic merge tools to detect and resolve dynamic semantics conflicts

But what exactly is a dynamic semantics conflict?

Undesired interference between integrated developers changes

G. Cavalcanti, P. Borba, P. Accioly. Evaluating and Improving Semistructured Merge. OOPSLA 2017.

But what exactly is interference?

Two procedures interfere when one can perform a global action which has a global effect up on the other

J. Reynolds. Syntactic Control of Interference.
POPL 1978.

One group of users, using a certain set of commands, is noninterfering with another group of users if what the first group does with these commands has no effect on what the second group of users can see

J. Goguen and J. Meseguer. Security Policies and Security Models. IEEE SSP 1982.

Separate changes L and R to a base program B interfere when the integrated changes does not preserve the changed behavior of L or R, or the unchanged behavior of B

S. Horwitz, J. Prins, and T. Reps. Integrating Noninterfering Versions of Programs. TOPLAS 1989 (POPL 1988).

J. Reynolds. Syntactic Control of Interference. POPL 1978.

J. Goguen and J. Meseguer. Security Policies and Security Models. IEEE SSP 1982.

We have interference because the integrated changes does not preserve the changed behavior of the yellow developer (version)

```
class Text {
  String text;
  …
  void cleanText() {
    normalizeWhitespace();
    removeComments();
    removeDuplicateWords();
  }
}
```

```
text = "the␣the␣␣dog"

text = "the␣the␣dog"

text = "the␣␣dog"
```

We have a semantic conflict because we have undesired interference,

assuming the intention captured in the boxes

resulting text has no duplicate whitespace

resulting text has no duplicate words

```
class Text {
  String text;
  …
  void cleanText() {
    normalizeWhitespace();
    removeComments();
    removeDuplicateWords();
  }
}
```

```
text = "the␣the␣␣dog"

text = "the␣the␣dog"

text = "the␣␣dog"
```

Partially expressing interference in terms of specifications

Separate changes L and R to a base program interfere if their behavior change specifications are not jointly satisfied by the program that integrates L and R

behavior change specifications constrain only the values of the state elements that have been affected by the change

```
{true}

normalizeWhitespace();
removeComments();

{no duplicate whitespace}
```

```
{true}

removeComments();
removeDuplicateWords();

{no duplicate words}
```



```
{true && true}

normalizeWhitespace();
removeComments();
removeDuplicateWords();

{no duplicate whitespace && no duplicate words}
```

{preL}

L

{postL}

{preR}

R

{postR}



{preL && preR}

merge(L,R,Base)

{postL && postR}

Implication, not equivalence: interference because the unchanged behavior of base is not preserved

```
{true}
x = 1;
skip;
y = 0;
if (x+y > 1) {z = 1;}
else {z = 0;}
{x == 1}
```

$z = 0$

```
{true}
x = 0;
skip;
y = 1;
if (x+y > 1) {z = 1;}
else {z = 0;}
{y == 1}
```



```
{true && true}
x = 1;
skip;
y = 1;
if (x+y > 1) {z = 1;}
else {z = 0;}
{x == 1 && y == 1}
```

$z = 1$

```
public class A{
        private final Map<String, String> tableTypes;

        public A(){
                tableTypeArrays = new HashMap<String, String>();
                tableTypes.put(“a”,”A”);
                tableTypes.put(“b”,”B”);
                tableTypes.put(“c”,”C”);
                tableTypes.put(“d”,”D”);
        }
}
```

```
public class A{
        private final Map<String, String> tableTypes;

        public A(){
                tableTypeArrays = new HashMap<String, String>();
                tableTypes.put(“a”,”A”);
                tableTypes.put(“b”,”B”);
                tableTypes.put(“c”,”C”);
                tableTypes.put(“d”,”D”);
        }
}
```

```
public class A{
        private final Map<String, String> tableTypes;

        public A(){
                tableTypeArrays = new HashMap<String, String>();
                tableTypes.put(“a”,”A”);
                tableTypes.put(“b”,”B”);
                tableTypes.put(“c”,”C”);
                tableTypes.put(“d”,”D”);
                tableTypes.put(“e”,”E”);
        }
}
```

Partially expressing interference in terms of more concrete conditions

Separate changes L and R to a base program B interfere if there is a state element x such that B, L and R compute different values for x

S. Horwitz, J. Prins, and T. Reps. Integrating Noninterfering Versions of Programs. TOPLAS 1989 (POPL 1988).

variable instead of state element

B, L and R compute different values for `Text.text`

```
class Text {
  String text;
  …
  void cleanText() {
    normalizeWhitespace();
    removeComments();
    removeDuplicateWords();
  }
}
```

`text = "the␣the␣␣dog"` `text = "the␣the␣dog"` `text = "the␣␣␣dog"`

Implication, not equivalence: there is no such x but there is interference

(unchanged behavior of base is not preserved)

```
x = 1;
skip;
y = 0;
if (x+y > 1) {z = 1;}
else {z = 0;}
```

z = 0

```
x = 0;
skip;
y = 1;
if (x+y > 1) {z = 1;}
else {z = 0;}
```

z = 0

```
x = 1;
skip;
y = 1;
if (x+y > 1) {z = 1;}
else {z = 0;}
```

z = 1

Separate changes L and R to a base program B interfere if there is a state element x such that L (or R) computes a different value for x than B and Merge

S. Horwitz, J. Prins, and T. Reps. Integrating Noninterfering Versions of Programs. TOPLAS 1989 (POPL 1988).

variable instead of state element

negation

L compute different value for `Text.text` than B and Merge compute

```
class Text {
  String text;
  …
  void cleanText() {
    normalizeWhitespace();
    removeComments();
    removeDuplicateWords();
  }
}
```

`text = "the_the__dog"` `text = "the_the_dog"` `text = "the__dog"`

Implication, not equivalence: there is no such x but there is interference

(unchanged behavior of base is not preserved)

```
x = 1;
skip;
y = 0;
if (x+y > 1) {z = 1;}
  else {z = 0;}
```

z = 0

```
x = 0;
skip;
y = 1;
if (x+y > 1) {z = 1;}
  else {z = 0;}
```

z = 0

```
x = 1;
skip;
y = 1;
if (x+y > 1) {z = 1;}
else {z = 0;}
```

z = 1

Separate changes L and R to a base program B interfere if there is a state element x such that B, L and R compute the same value for x but Merge computes a different value

S. Horwitz, J. Prins, and T. Reps. Integrating Noninterfering Versions of Programs. TOPLAS 1989 (POPL 1988).

variable instead of state element

negation

B, L and R compute the same value for z but Merge computes a different value

```
x = 1;
skip;
y = 0;
if (x+y > 1) {z = 1;}
  else {z = 0;}
```

z = 0

```
x = 0;
skip;
y = 1;
if (x+y > 1) {z = 1;}
  else {z = 0;}
```

z = 0

```
x = 1;
skip;
y = 1;
if (x+y > 1) {z = 1;}
else {z = 0;}
```

z = 1

Separate changes L and R to a base program B interfere if Merge has new effects besides the ones in L and R

Same property, but expressed in terms of effects instead of concrete state elements

Detailing the auxiliary definitions

Analyzing methods with an open world perspective

State element stands for any part of the system state (global variables, object fields, system files, output stream, visible GUI elements, variables that hold method return values or raised exceptions, etc.)

Analyzing methods with a restricted open world (API) perspective

State element stands for any part of the system state (global variables, object fields, system files, output stream, visible GUI elements, variables that hold method return values or raised exceptions, etc.) that affect the execution of other methods of the same API

Analyzing the system with a closed world perspective

State element stands for any part of the system state that is externally visible during and after program execution (system files, output stream, visible GUI elements, etc.)

Computes captures reachability. So integrated changes might involve only statements with y, but end up impacting the final computed value of x

Assumptions

- Sequential execution
- No observation of system resources (CPU time, memory, power consumption, etc.) usage
- Changes that do not affect interfaces

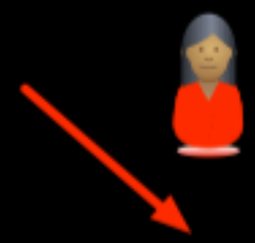
But interference is not computable!

Are there approximations for automatically detecting interference (and semantic conflicts)?

```
class Text {
  public String text;
  …
  void cleanText() {
    removeComments();
  }
}
```



```
class Text {
  public String text;
  …
  void cleanText() {
    removeComments();
    normalizeWhitespace();
  }
}
```



```
class Text {
  public String text;
  …
  void cleanText() {
    removeDuplicateWords();
    removeComments();
  }
}
```

merge

```
class Text {
  public String text;
  …
  void cleanText() {
    removeDuplicateWords();
    removeComments();
    normalizeWhitespace();
  }
}
```

Interference and semantic conflicts